

STK1110

Ana María Jiménez Güiza

September 2026

Innhold

Oppgave 1	1
Oppgave 2	8
R kode for oppgave 1	14
R kode for oppgave 2	16

Oppgave 1

(a) Utled moment-estimatorene for α og γ . Gitt moment-estimatorene for α og β (gitt i boka), er disse resultatene rimelige?

Vi lar gjennomsnittet fra data være et estimat for forventningsverdien i modellen, da har vi:

$$\bar{X} = E[X]$$

Fra oppgave teksten vet vi at

$$E[X] = \frac{\alpha}{\gamma}$$

så vi får

$$\bar{X} = \frac{\alpha}{\gamma}$$

Vi skriver variansen som

$$S^2 \approx V[X] = \frac{\alpha}{\gamma^2}$$

Vi løser diss to ligningene med hensyn på α og γ , så får vi

$$\alpha = \bar{X}\gamma$$

og sette dette i variansen

$$S^2 = \frac{\alpha}{\gamma^2} = \frac{\bar{X}\gamma}{\gamma^2} = \frac{\bar{X}}{\gamma}$$

Løser vi for γ

$$\gamma = \frac{\bar{X}}{S^2}$$

Beregn momentestimatorene basert på forsikringsdataene.

I R kan vi finne verdiene for $\bar{X}$ og S^2

```
1 #Oppgave 1a
2 x = read.table("https://www.uio.no/studier/emner/matnat/math/STK_
   ↪ 1110/data/forsikringskrav.txt", header=F)[,1]
3
4 n = length(x)
5 mean(x)
6 var(x)
7
8 # Moment-estimatorene for alpha og gamma
9 gamma.mom = mean(x)/var(x)
10 alpha.mom = mean(x) * gamma.mom
11
12 alpha.mom
13 gamma.mom
```

Vi får

$$\bar{X} = 24.13847$$

og

$$S^2 = 836.5148$$

Setter vi disse i de uttrykkene vi regnet ut så har vi

$$\hat{\alpha} = \bar{X}\gamma \approx 0.69654$$

og

$$\hat{\gamma} = \frac{\bar{X}}{S^2} \approx 0.028856$$

(b) Skriv ned log-likelihood funksjonen og beregn log-likelihood verdien basert på dine estimater for α, γ

Likelihood funksjonen er

$$L(\alpha, \gamma) = \prod_{i=1}^n f(x_i; \alpha, \gamma)$$

og log-likelihood er

$$\ell(\alpha, \gamma) = \log L(\alpha, \gamma)$$

Fra oppgavetekst vi har

$$f(x; \alpha, \gamma) = \frac{\gamma^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\gamma x}$$

Vi bruker log regler, og kan derfor skrive log-likelihood som

$$\ell(\alpha, \gamma) = \sum_{i=1}^n [\alpha \log(\gamma) - \log(\Gamma(\alpha)) + (\alpha - 1) \log(x_i) - \gamma x_i]$$

Dette kan vi forkorte til

$$\ell(\alpha, \gamma) = n\alpha \log(\gamma) - n \log(\Gamma(\alpha)) + (\alpha - 1) \sum_{i=1}^n \log(x_i) - \gamma \sum_{i=1}^n x_i$$

Vi bruker R for å beregne log-likelihood

```

15 # Oppgave 1b
16 loglik = n * alpha.mom * log(gamma.mom) -
17   n * lgamma(alpha.mom) +
18   (alpha.mom - 1) * sum(log(x)) -
19   gamma.mom * sum(x)
20
21 loglik

```

Da får vi $\ell(\hat{\alpha}, \hat{\gamma}) \approx -27264.84$

(c) La oss se på maksimum likelihood estimatorene for α, γ . Her eksisterer det ikke noe analytisk uttrykk. Vis imidlertid at for gitt α , så er det et analytisk uttrykk for maksimum likelihood estimatoren for γ (som en funksjon av α)

Maksimum likelihood-estimator for γ når α betraktes som kjent, kan regnes som

$$\frac{\partial}{\partial \gamma} \ell(\alpha, \gamma) = \frac{n\alpha}{\gamma} - \sum_{i=1}^n x_i$$

For å finne MLE $\hat{\gamma}$ setter vi den deriverte lik null, og løser for $\hat{\gamma}$

$$\frac{n\alpha}{\gamma} - \sum_{i=1}^n x_i = 0$$

som gir

$$\frac{\alpha}{\gamma} = \frac{1}{n} \sum_{i=1}^n x_i = \bar{x}$$

dermed har vi

$$\hat{\gamma}_{ML} = \frac{\alpha}{\bar{x}}$$

Utledd dette uttrykket. Vis at dette svarer til moment-estimatoren for γ (for gitt α)

Når α er fiksert, gir momentmetoden:

$$\bar{X} = E[X] = \frac{\alpha}{\gamma}$$

Løser vi for γ , får vi

$$\hat{\gamma}_{moment}(\alpha) = \frac{\alpha}{\bar{X}}$$

og fra maksimum likelihood fikk vi nøyaktig det samme

$$\hat{\gamma}_{ML}(\alpha) = \frac{\alpha}{\bar{X}}$$

Som vil si at

$$\hat{\gamma}_{moment} = \hat{\gamma}_{ML}$$

(d) Forklar hvorfor en bør ha $\log(\alpha)$ som input til funksjonen og ikke α selv. Bruk denne funksjonen til å finne maksimum likelihood estimator for (α, γ) . Beregn log-likelihood verdien for dette settet av estimator. Er denne verdien rimelig i forhold til den verdi du fikk i (b)?

Man bør ha $\log(\alpha)$ som input siden for gammafordelingene gjelder at $\alpha > 0$. Ved å bruke

$$\alpha = \exp(\log(\alpha))$$

sikrer vi oss at dette gjelder hver gang uansett hvilken verdi av $\log(\alpha)$ optimeringsalgoritmen prøver.

Ved bruk av koden i oppgaveteksten får vi

$$par = 0.3281129 = \log(\hat{\alpha})$$

og

$$value = 26488.28 = -\ell(\hat{\alpha}, \hat{\gamma})$$

Vi bruker den følgende kode for å finne verdiene av våre estimator:

```
23 # Oppgave 1d
24 negloglikgamma = function(logalpha,x=x)
25 {
26   n = length(x)
```

```

27   alpha=exp(logalpha)
28   gamma = alpha/mean(x)
29   logL = n*alpha*log(gamma)-n*lgamma(alpha)+
30         (alpha-1)*sum(log(x))-gamma*sum(x)
31   -logL
32 }
33
34 fit.ml=optim(log(alpha.mom),negloglikgamma,x=x,method="BFGS")
35 fit.ml
36
37 # ML-estimatorene for alpha og gamma
38 alpha.ml = exp(fit.ml$par)
39 gamma.ml = alpha.ml / mean(x)
40
41 alpha.ml
42 gamma.ml

```

Da får vi

$$\hat{\alpha}_{ML} = 1.388346 \quad \hat{\gamma}_{ML} = 0.05751589$$

Log-likelihooden ved ML-estimaten er -26488.28, mens den var -27264.84 ved momentestimaten i 1b. ML-estimaten gir altså en høyere log-likelihood. Dette er rimelig, siden ML-estimaten er valgt for å maksimere likelihooden.

(e) Basert på ikke-parametrisk bootstrapping, finn anslag på standard feil for maksimum likelihood estimaten for α og γ .

Vi bruker følgende kode for å bootstrappe $\hat{\alpha}$ og $\hat{\gamma}$. Deretter finner vi standardfeilene som standardavviket til de 1000 bootstrap-estimaten.

```

44 # Oppgave 1e
45 B = 1000
46 alpha.boot = numeric(B)
47 gamma.boot = numeric(B)
48
49 for (b in 1:B) {
50   # Trekk et bootstrap-datasett
51   x.boot = sample(x, size = length(x), replace = T)
52
53   # Finn ML-estimatet
54   fit.boot = optim(log(alpha.ml), negloglikgamma, x = x.boot,
55     ↪ method = "BFGS")
56
57   # Lagre estimatet for alpha og gamma
58   alpha.boot[b] = exp(fit.boot$par)

```

```

58   gamma.boot[b] = alpha.boot / mean(x.boot)
59 }
60
61 # Estimering av standardavviket til bootstrap-estimatene
62 sd(alpha.boot)
63 sd(gamma.boot)

```

Dette gir oss

$$\widehat{SE}(\hat{\alpha}_{ML}) = 0.03001 \quad \text{og} \quad \widehat{SE}(\hat{\gamma}_{ML}) = 0.000879$$

Bruk dette til å lage 95% konfidens-intervaller for α og γ .

Vi bruker koden gitt i oppgaveteksten, og verdiene til $\hat{\alpha}_{boot}$ og $\hat{\gamma}_{boot}$ for å finne nedre og øvre grensene for estimatene.

```

65 # 95% konfidens-intervaller for alpha og gamma
66 quantile(alpha.boot, c(0.025, 0.975))
67 quantile(gamma.boot, c(0.025, 0.975))

```

Dette gir oss

$$CI_{95\%}(\alpha) \approx (1.33401, 1.45352)$$

og

$$CI_{95\%}(\gamma) \approx (0.05468, 0.05787)$$

(f) Anta nå vi er mer interessert i $E[X_i] = \mu = \alpha/\gamma$. Bruk dine bootstrap simuleringer til å lage et 95% konfidensintervall for μ .

Siden $\mu = \alpha/\gamma$, kan vi bruke $\hat{\alpha}_{boot}$ og $\hat{\gamma}_{boot}$ for å regne $\hat{\mu}_{boot}$. Videre kan vi bruke $\hat{\mu}_{boot}$ for å lage et 95% konfidensintervall for μ .

```

69 # Oppgave 1f
70
71 # mu = alpha / gamma
72 mu.boot = alpha.boot / gamma.boot
73
74 # 95% konfidens-intervall for mu
75 quantile(mu.boot, c(0.025, 0.975))

```

Dette gir oss

$$CI_{95\%}(\mu) = (23.94914, 25.47975)$$

Anta vi ønsker å teste hypotesene

$$H_0 : \mu = 25 \text{ mot } H_a : \mu \neq 25$$

med et signifikansnivå $\alpha = 0.05$. Hva blir konklusjonen på en slik test?

Et signifikansnivå $\alpha = 0.05$ tilsvarer et 95% konfidensintervall. Om $\mu = 25$ forkaster vi ikke H_0 siden 25 ligger innenfor intervallet.

Hvis du endrer signifikansnivået til $\alpha = 0.01$, hva blir så konklusjonen?

Et signifikansnivå $\alpha = 0.01$ tilsvarer et 99% konfidensintervall. Dette kan vi regne med følgende kode

```
77 # 99% konfidens-intervall for mu
78 quantile(mu.boot, c(0.005, 0.995))
```

Som gir oss

$$CI_{99\%}(\mu) = (23.68342, 25.69306)$$

Om $\mu = 25$ forkaster vi heller ikke H_0 siden 25 ligger innenfor intervallet.

Basert på dette, hva kan du si om P-verdien for testen?

Siden 25 ligger innenfor både 95% og 99% konfidensintervallet, forkastes ikke $H_0 : \mu = 25$ ved signifikansnivå 0.05 eller 0.01. Dette tilsier at P-verdien er større enn eller lik 0.05.

Oppgave 2

(a) Lag et 95% konfidensintervall for forventet antocyaninnhold μ basert på målingene fra oppgaveteksten.

Siden målingene antas å være uavhengige og normalfordelte og variansen er ukjent, bruker vi et t-intervall for μ . Som er

$$\bar{X} \pm t_{n-1, 0.975} \frac{S}{\sqrt{n}}$$

I R kan vi bruke følgende kode for å finne t-verdien

```
1 # Oppgave 2a
2 x = c(525, 587, 547, 558, 591, 531, 571, 551, 566, 622, 561,
      ↪ 502, 556, 565, 562)
3 n = length(x)
4
5 #t-verdien for 95% konfidensintervall
6 t = qt(0.975, df = n-1)
```

Da har vi

$$t_{14, 0.975} = 2.144787$$

Vi trenger også å regne standardfeilen til gjennomsnittet

```
8 # Standardfeilen til gjennomsnittet
9 SE = sd(x)/sqrt(n)
```

Da har vi

$$SE = \frac{s}{\sqrt{n}} = 7.374129$$

Da kan vi regne feilmarginen og derfor også intervallet.

```
11 # Feilmarginen
12 fm = t * SE
13
14 #Øvre og nedre grense av intervallet
15 øvre = mean(x) + fm
16 nedre = mean(x) - fm
```

som gir 95% konfidensintervallet

$$(543.85, 575.48)$$

(b) På Wikipedia kan vi lese at forventet antocyaninnhold i blåbær er 558 mg/100g. Nå skal du bruke simuleringer til å late som om du måler antocyan i 15 prøver med blåbær veldig mange ganger. Generér 10000 datasett, hvert av størrelse $n = 15$.

Lag et 95% konfidensintervall for μ som i punkt (a), basert på hvert av de simulerte datasettene, slik at du får 10000 intervaller. Tell opp andelen av disse intervallene som inneholder verdien 558. Kommentér og forklar.

I den følgende kode har vi laget et simulasjon som generer 10 000 datasett, hver med lengde 15. Deretter finner det øvre og nedre grenser for å lage 95% konfidensintervallet for μ .

```

18 # Oppgave 2b
19 B = 10000
20 nedre = numeric(B)
21 øvre = numeric(B)
22
23 # Simulasjon for 10 000 datasett
24 for (i in 1:B) {
25   x = rnorm(15, mean = 558, sd = 30)
26
27   x.bar = mean(x)
28   s = sd(x)
29   SE = s/sqrt(15)
30
31   t = qt(0.975, df = 14)
32
33   # Finn øvre og nedre grense for intervallene
34   øvre[i] = x.bar + t*SE
35   nedre[i] = x.bar - t*SE
36 }

```

Den følgende kode regner andelen av hvor mange intervaller inneholder verdien 558.

```

38 # Intervallet inneholder 558
39 inneholder = (nedre <= 558) & (øvre >= 558)
40 mean(inneholder)

```

I dette tilfellet får vi 0.9489, altså 94.89% av konfidensintervallene inneholdte $\mu = 558$. Dette er tilstrekkelig nær til det vi forventet med en 95% konfidensintervall. Det vil si, hvis vi gjentar forsøket mange ganger, forventer vi at omtrent 95% av intervallene inneholder den sanne forventningsverdien μ .

(c) Bruk nå i stedet det tilnærmede intervallet for store utvalg, altså

$$\left(\bar{X} - 1.96 \frac{S}{\sqrt{15}}, \bar{X} + 1.96 \frac{S}{\sqrt{15}} \right)$$

med

$$\bar{X} = \frac{1}{15} \sum_{i=1}^{15} X_i \quad \text{og} \quad S^2 = \frac{1}{15-1} \sum_{i=1}^{15} (X_i - \bar{X})^2$$

og beregn dette for hvert av 10000 datasett, generert som i (b). Hvor stor andel av intervallene inneholder $\mu = 558$? Kommentér og forklar resultatet.

Vi bruker det samme kode som vi brukte i (b), med unntak på at vi endre t-verdien med 1.96.

```

42 # Oppgave 2c
43 B = 10000
44 nedre.c = numeric(B)
45 øvre.c = numeric(B)
46
47 # Simulasjon for 10 000 datasett
48 for (i in 1:B) {
49   x = rnorm(15, mean = 558, sd = 30)
50
51   x.bar = mean(x)
52   s = sd(x)
53   SE = s/sqrt(15)
54
55   # Finn øvre og nedre grense for intervallene
56   øvre.c[i] = x.bar + 1.96*SE
57   nedre.c[i] = x.bar - 1.96*SE
58 }
59
60 # Intervallet inneholder 558
61 inneholder.c = (nedre.c <= 558) & (øvre.c >= 558)
62 mean(inneholder.c)

```

I denne tilfelle får vi 0.9308, altså 93.08%. Siden nå ganger vi standardfeilen med et lavere tall, det er ikke overraskende at vi får også et lavere andel. Dette er pga. intervallet blir smalere, og at t-verdien klarer å ta bedre hensyn når n er liten.

(d) Trekk 10000 datasett som i (b) og lag et 95% konfidensintervall for σ for hvert av dem. Hvor stor andel av intervallene inneholder $\sigma = 30$?

For dette trenger vi χ^2 -fordelingen, og vi vet at vi kan da finne 95% konfidensintervall for σ med

$$\left(S\sqrt{\frac{n-1}{\chi_{0.975,n-1}^2}}, S\sqrt{\frac{n-1}{\chi_{0.025,n-1}^2}} \right)$$

Vi først regner $\chi_{0.975,14}^2$ og $\chi_{0.025,14}^2$ med den følgende kode

```

64 # Oppgave 2d
65 chi.nedre = qchisq(0.025, df = 14)
66 chi.øvre = qchisq(0.975, df = 14)

```

Dette gir oss

$$\chi^2_{0.975,14} = 26.11895 \quad \text{og} \quad \chi^2_{0.025,14} = 5.628726$$

Da kan vi bruke liknende kode som i oppgavene (b) og (c) for å regne konfidensintervallet, og finne andel for at intervallet inneholder $\sigma = 30$

```
64 # Oppgave 2d
65 chi.nedre = qchisq(0.025, df = 14)
66 chi.øvre = qchisq(0.975, df = 14)
67
68 B = 10000
69 nedre.sigma = numeric(B)
70 øvre.sigma = numeric(B)
71
72 # Simulasjon for 10 000 datasett
73 for (i in 1:B) {
74   x = rnorm(15, mean = 558, sd = 30)
75   s = sd(x)
76
77   # Finn øvre og nedre grense for intervallene
78   øvre.sigma[i] = s*(sqrt((n-1)/chi.nedre))
79   nedre.sigma[i] = s*(sqrt((n-1)/chi.øvre))
80 }
81
82 # Intervallet inneholder sigma = 30
83 inneholder.sigma = (nedre.sigma <= 30) & (øvre.sigma >= 30)
84 mean(inneholder.sigma)
```

Dette gir oss 0.9513, altså 95.13% av intervallene inneholder $\sigma = 30$.

(e) Under antakelsen om normalfordeling er $Z_i = \frac{X_i - \mu}{\sigma} \sim N(0, 1)$, $i = 1, \dots, n$, med $\mu = 558$ og $\sigma = 30$. Anta nå at $Z_1, \dots, Z_{15}$ i virkeligheten er t-fordelt med 7 frihetsgrader, altså $Z_1, \dots, Z_{15} \stackrel{uif}{\sim} t_7$. Trekk nå 10000 datasett fra denne fordelingen ved å bruke R-funksjonen `rt()`. Gjenta deretter oppgave (b) med de nye datasettene. Hvor robust er metoden for å lage konfidensintervall for forventningsverdien for antakelsen om normalfordeling?

For å kunne lage datasettet trenger vi først å løse for X_i

$$X_i = \mu + \sigma Z_i = 558 + 30Z_i$$

Da kan vi gjenta samme kode som i oppgave 2b. Legg merke at vi først finner Z_i med 7 frihetsgrader, så lager vi x_i ut av verdiene.

```
86 # Oppgave 2e
87 B = 10000
```

```

88 nedre = numeric(B)
89 øvre = numeric(B)
90
91 # Simulasjon for 10 000 datasett
92 for (i in 1:B) {
93   z = rt(15, df = 7)
94   x = 558 + 30*z
95
96   x.bar = mean(x)
97   s = sd(x)
98   SE = s/sqrt(15)
99
100   t = qt(0.975, df = 14)
101
102   # Finn øvre og nedre grense for intervallene
103   øvre[i] = x.bar + t*SE
104   nedre[i] = x.bar - t*SE
105 }
106
107 # Intervallet inneholder mu = 558
108 inneholder = (nedre <= 558) & (øvre >= 558)
109 mean(inneholder)

```

Dette ga oss 0.9497, altså 94.95%. Dette er veldig nær den ønskede dekningsgraden på 95%, og veldig nær til resultatet i 2b. Derfor er metoden ganske robust.

(f) Trekk datasett som i (e) og lag deretter 95% konfidensintervall for standardavviket til X_i slik som i (d). Merk imidlertid at $V(Z_i) = \frac{7}{7-2}$ slik at variansen til X_i nå er $\tilde{\sigma}^2 = V(X_i) = V(\mu + \sigma Z_i) = \sigma^2 V(Z_i) = 1.4\sigma^2$. Det er altså $\tilde{\sigma}$ du skal lage konfidensintervall for, og sjekke andelen intervaller som inneholder $\tilde{\sigma}$. Sammenlign med resultatene fra (d) og kommentér.

Først kan vi løse for $\tilde{\sigma}$, da har vi

$$\tilde{\sigma} = \sigma\sqrt{1.4}$$

Siden vi bruker $\sigma = 30$, kan vi også regne $\tilde{\sigma} = 30\sqrt{1.4} \approx 35.50$

Da kan vi bruke en sammensetting av kodene vi brukte i 2d og 2e. Dvs. generere data med `rt()`, og lage konfidensintervallet for standardavviket med kjikvadratfordelingen. Legg merke at nå sjekker vi om $30 * \sqrt{1.4}$ er i intervallet i stedet for 30 som i 2b.

```

111 # Oppgave 2f
112 chi.nedre = qchisq(0.025, df = 14)
113 chi.øvre = qchisq(0.975, df = 14)
114

```

```

115 B = 10000
116 nedre.sigma = numeric(B)
117 øvre.sigma = numeric(B)
118
119 # Det faktiske standardavviket til X
120 sigma.tilde = 30*sqrt(1.4)
121
122 # Simulasjon for 10 000 datasett
123 for (i in 1:B) {
124   z = rt(15, df = 7)
125   x = 558 + 30*z
126   s = sd(x)
127
128   # Finn øvre og nedre grense for intervallene
129   øvre.sigma[i] = s*(sqrt((n-1)/chi.nedre))
130   nedre.sigma[i] = s*(sqrt((n-1)/chi.øvre))
131 }
132
133 inneholder = (nedre.sigma <= sigma.tilde) & (vre.sigma >=
134   ↪ sigma.tilde)
135 mean(inneholder)

```

Da får vi 0.8878, altså 88,78%. Dette er mye lavere enn i 2d. Metoden for å lage konfidensintervallet for standardavviket mindre robust mot avvik fra normalfordeling. Siden dataene nå er basert på en t_7 -fordeling, er antakelsen om at observasjonene er normalfordelte ikke oppfylt.

R kode for oppgave 1

```
1  #Oppgave 1a
2  x = read.table("https://www.uio.no/studier/emner/matnat/math/STK_
   ↪ 1110/data/forsikringskrav.txt", header=F)[,1]
3
4  n = length(x)
5  mean(x)
6  var(x)
7
8  # Moment-estimatorene for alpha og gamma
9  gamma.mom = mean(x)/var(x)
10 alpha.mom = mean(x) * gamma.mom
11
12 alpha.mom
13 gamma.mom
14
15 # Oppgave 1b
16 loglik = n * alpha.mom * log(gamma.mom) -
17   n * lgamma(alpha.mom) +
18   (alpha.mom - 1) * sum(log(x)) -
19   gamma.mom * sum(x)
20
21 loglik
22
23 # Oppgave 1d
24 negloglikgamma = function(logalpha,x=x)
25 {
26   n = length(x)
27   alpha=exp(logalpha)
28   gamma = alpha/mean(x)
29   logL = n*alpha*log(gamma)-n*lgamma(alpha)+
30     (alpha-1)*sum(log(x))-gamma*sum(x)
31   -logL
32 }
33
34 fit.ml=optim(log(alpha.mom),negloglikgamma,x=x,method="BFGS")
35 fit.ml
36
37 # ML-estimatorene for alpha og gamma
38 alpha.ml = exp(fit.ml$par)
39 gamma.ml = alpha.ml / mean(x)
40
41 alpha.ml
42 gamma.ml
43
```

```

44 # Oppgave 1e
45 B = 1000
46 alpha.boot = numeric(B)
47 gamma.boot = numeric(B)
48
49 for (b in 1:B) {
50   # Trekk et bootstrap-datasett
51   x.boot = sample(x, size = length(x), replace = T)
52
53   # Finn ML-estimatet
54   fit.boot = optim(log(alpha.ml), negloglikgamma, x = x.boot,
55     ↪ method = "BFGS")
56
57   # Lagre estimatet for alpha og gamma
58   alpha.boot[b] = exp(fit.boot$par)
59   gamma.boot[b] = alpha.boot / mean(x.boot)
60 }
61
62 # Estimering av standardavviket til bootstrap-estimatene
63 sd(alpha.boot)
64 sd(gamma.boot)
65
66 # 95% konfidens-intervaller for alpha og gamma
67 quantile(alpha.boot, c(0.025, 0.975))
68 quantile(gamma.boot, c(0.025, 0.975))
69
70 # Oppgave 1f
71
72 #  $\mu = \alpha / \gamma$ 
73 mu.boot = alpha.boot / gamma.boot
74
75 # 95% konfidens-intervall for  $\mu$ 
76 quantile(mu.boot, c(0.025, 0.975))
77
78 # 99% konfidens-intervall for  $\mu$ 
79 quantile(mu.boot, c(0.005, 0.995))

```

R kode for oppgave 2

```
1  # Oppgave 2a
2  x = c(525, 587, 547, 558, 591, 531, 571, 551, 566, 622, 561,
   ↪ 502, 556, 565, 562)
3  n = length(x)
4
5  #t-verdien for 95% konfidensintervall
6  t = qt(0.975, df = n-1)
7
8  # Standardfeilen til gjennomsnittet
9  SE = sd(x)/sqrt(n)
10
11 # Feilmarginen
12 fm = t * SE
13
14 #Øvre og nedre grense av intervallet
15 øvre = mean(x) + fm
16 nedre = mean(x) - fm
17
18 # Oppgave 2b
19 B = 10000
20 nedre = numeric(B)
21 øvre = numeric(B)
22
23 # Simulasjon for 10 000 datasett
24 for (i in 1:B) {
25   x = rnorm(15, mean = 558, sd = 30)
26
27   x.bar = mean(x)
28   s = sd(x)
29   SE = s/sqrt(15)
30
31   t = qt(0.975, df = 14)
32
33   # Finn øvre og nedre grense for intervallene
34   øvre[i] = x.bar + t*SE
35   nedre[i] = x.bar - t*SE
36 }
37
38 # Intervallet inneholder mu = 558
39 inneholder = (nedre <= 558) & (øvre >= 558)
40 mean(inneholder)
41
42 # Oppgave 2c
43 B = 10000
```

```

44 nedre.c = numeric(B)
45 øvre.c = numeric(B)
46
47 # Simulasjon for 10 000 datasett
48 for (i in 1:B) {
49   x = rnorm(15, mean = 558, sd = 30)
50
51   x.bar = mean(x)
52   s = sd(x)
53   SE = s/sqrt(15)
54
55   # Finn øvre og nedre grense for intervallene
56   øvre.c[i] = x.bar + 1.96*SE
57   nedre.c[i] = x.bar - 1.96*SE
58 }
59
60 # Intervallet inneholder mu = 558
61 inneholder.c = (nedre.c <= 558) & (øvre.c >= 558)
62 mean(inneholder.c)
63
64 # Oppgave 2d
65 chi.nedre = qchisq(0.025, df = 14)
66 chi.øvre = qchisq(0.975, df = 14)
67
68 B = 10000
69 nedre.sigma = numeric(B)
70 øvre.sigma = numeric(B)
71
72 # Simulasjon for 10 000 datasett
73 for (i in 1:B) {
74   x = rnorm(15, mean = 558, sd = 30)
75   s = sd(x)
76
77   # Finn øvre og nedre grense for intervallene
78   øvre.sigma[i] = s*(sqrt((n-1)/chi.nedre))
79   nedre.sigma[i] = s*(sqrt((n-1)/chi.øvre))
80 }
81
82 # Intervallet inneholder sigma = 30
83 inneholder.sigma = (nedre.sigma <= 30) & (øvre.sigma >= 30)
84 mean(inneholder.sigma)
85
86 # Oppgave 2e
87 B = 10000
88 nedre = numeric(B)
89 øvre = numeric(B)

```

```

90
91 # Simulasjon for 10 000 datasett
92 for (i in 1:B) {
93   z = rt(15, df = 7)
94   x = 558 + 30*z
95
96   x.bar = mean(x)
97   s = sd(x)
98   SE = s/sqrt(15)
99
100   t = qt(0.975, df = 14)
101
102   # Finn øvre og nedre grense for intervallene
103   øvre[i] = x.bar + t*SE
104   nedre[i] = x.bar - t*SE
105 }
106
107 # Intervallet inneholder mu = 558
108 inneholder = (nedre <= 558) & (øvre >= 558)
109 mean(inneholder)
110
111 # Oppgave 2f
112 chi.nedre = qchisq(0.025, df = 14)
113 chi.øvre = qchisq(0.975, df = 14)
114
115 B = 10000
116 nedre.sigma = numeric(B)
117 øvre.sigma = numeric(B)
118
119 # Det faktiske standardavviket til X
120 sigma.tilde = 30*sqrt(1.4)
121
122 # Simulasjon for 10 000 datasett
123 for (i in 1:B) {
124   z = rt(15, df = 7)
125   x = 558 + 30*z
126   s = sd(x)
127
128   # Finn øvre og nedre grense for intervallene
129   øvre.sigma[i] = s*(sqrt((n-1)/chi.nedre))
130   nedre.sigma[i] = s*(sqrt((n-1)/chi.øvre))
131 }
132
133 inneholder = (nedre.sigma <= sigma.tilde) & (vre.sigma >=
  ↪ sigma.tilde)
134 mean(inneholder)

```